

Esame OS Internals (Cabodi)

Data: 02/09/2024

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale e IPT)

Q1) Rispondere YES/NO a ognuna delle domande seguenti:

1. In un sistema di memoria virtuale con un grado fisso di multiprogrammazione, il thrashing può verificarsi anche se la somma dei working set di tutti i processi è inferiore alla memoria fisica disponibile, ma l'algoritmo di sostituzione della pagina non è ottimizzato?
2. In un sistema con paginazione basata su inverted page table (IPT), è possibile che più pagine virtuali di processi diversi condividano una singola entry della tabella delle pagine da senza causare un conflitto nella traduzione degli indirizzi?
3. In un sistema di paginazione a richiesta che utilizza la sostituzione delle pagine locale (a un singolo processo), un processo con frequenza di page fault molto elevata può comunque causare una riduzione della memoria fisica disponibile per altri processi?

(Nota: Per ognuna delle risposte alle domande precedenti, occorre fornire una breve motivazione nella domanda successiva Q2).

Q2) Calcoli su IPT:

Si consideri un sistema a 64 bit con uno spazio di indirizzi virtuale di 2^{48} Byte, una dimensione di pagina di 4 KB e 16 GB di memoria fisica. Si supponga che il sistema utilizzi una inverted page table (IPT). Calcolare e spiegare quanto segue:

1. Il numero di bit necessari per il numero di frame fisico.
 2. Il numero totale di entry nella inverted page table.
 3. L'indirizzo virtuale assegnato a un processo è `0x00007FFFFFFFFF000`. Determinare l'indirizzo fisico se questo indirizzo virtuale è mappato al frame fisico 1024. Mostrare tutti i passaggi, incluso il calcolo del numero di pagina e dell'offset.
-

Domanda 2 (File System e Allocazione Linked)

Q3) Allocazione file:

Si consideri un file system che utilizza l'allocazione di tipo linked per l'allocazione dei file. Il disco è diviso in blocchi di 4 KB ciascuno. Ogni blocco contiene un puntatore al blocco successivo, che occupa 4 byte del blocco. Un file richiede 5 MB di spazio di archiviazione.

- A. Calcolare il numero di blocchi del disco necessari per allocare l'intero file, incluso lo spazio necessario per i puntatori.

- A. Se i puntatori sono stati ridotti a 2 byte (presupponendo una capacità del disco inferiore), ricalcolare il numero di blocchi necessari. Questa modifica renderebbe l'archiviazione dei file più efficiente in termini di utilizzo dei blocchi?
 - A. Confrontare il sovraccarico introdotto dai puntatori in entrambi i casi come percentuale dello spazio di archiviazione totale usato.
-

Domanda 3 (I/O, Interrupt, DMA e HDD)

Q4) Rispondere YES/NO a ognuna delle domande seguenti:

1. In un sistema con più dispositivi di I/O e una singola CPU, le operazioni di I/O possono essere eseguite in parallelo se la CPU è occupata nell'esecuzione di un processo?
2. In un sistema che utilizza I/O basato su interrupt, è possibile che un interrupt venga perso se il controller di interrupt è occupato nell'elaborazione di un altro interrupt e il dispositivo che ha attivato il secondo interrupt non supporta l'accodamento degli interrupt?
3. In un sistema con memoria virtuale basata su paginazione a richiesta e DMA, è possibile che si verifichi un page fault durante un trasferimento DMA, causando l'esito negativo del trasferimento se non vengono prese le dovute precauzioni?

(Nota: Per ognuna delle risposte alle domande precedenti, occorre fornire una breve motivazione nella domanda successiva Q6).

Q5) Calcoli su HDD:

Prendere in considerazione un'unità disco rigido (HDD) con le seguenti specifiche: Dimensione del settore: 512 byte; Numero di tracce per faccia: 5.000; Numero di settori per traccia: 300; Numero di piatti a doppia faccia: 6; Velocità di rotazione del piatto: 7.200 giri/min (giri al minuto).

1. Calcolare la velocità di trasferimento dati massima possibile in megabyte al secondo (MB/s), supponendo che sia possibile trasferire una traccia di dati per giro.
2. Se il disco subisce un arresto anomalo della testina su un piatto, in che modo ciò influisce sulla capacità totale e sulla disponibilità dei dati supponendo che non siano in atto meccanismi RAID o di backup?
3. Se il disco ha un tempo di ricerca medio di 4 ms e deve leggere un file da 1 GB suddiviso in 200 tracce non contigue, calcolare il tempo totale necessario per leggere il file. Includi il tempo per la ricerca, la latenza rotazionale e il trasferimento dei dati. Si supponga che la latenza rotazionale media sia di 4,165 ms e che sia possibile leggere una traccia per giro.

2 SECTION 2: OS161

Domanda 4 (dumbvm e Address Translation)

Contesto: Si consideri un sistema OS161, in esecuzione su un simulatore MIPS sys161 con 8 MB di memoria RAM.

Q6) Funzione `ram_stealmem`:

La memoria fisica in `dumbvm` viene allocata chiamando la funzione `ram_stealmem`. Per ciascuna delle seguenti frasi, rispondere vero/falso (true/false: non è necessaria alcuna spiegazione):

- Sebbene la soluzione proposta per supportare la deallocazione della memoria (`freeppages`) sia basata su una bitmap, potrebbe essere utilizzata anche una lista concatenata.
- La funzione `ram_stealmem` alloca in modo incrementale la memoria a partire dal primo indirizzo disponibile dopo aver fatto load del kernel.
- Quando si utilizza una bitmap per tenere traccia della RAM allocata/libera, è necessario un secondo array che memorizzi le dimensioni delle partizioni allocate per l'allocazione sia della memoria dinamica del kernel che dei processi utente.
- Lo stack di un processo utente non viene allocato (in `dumbvm`) chiamando `ram_stealmem`, perché la dimensione dello stack è una costante e lo stack si trova sempre agli stessi indirizzi logici.
- La funzione `ram_stealmem` viene chiamata per l'allocazione della memoria sia del kernel (memoria dinamica) che dei processi user.
- Lo schema di allocazione utilizzato da `ram_stealmem` non consentirà l'intercalare partizioni contigue del kernel e dei processi user (in altre parole, le partizioni del kernel e le partizioni user non saranno mai interlacciate).

Q7) Traduzione Indirizzi:

È noto che uno spazio di indirizzamento utente è caratterizzato da `as->as_pbase1`, `as->as_pbase2`, `as->as_vbase1`, `as->as_vbase2`, `as->as_npages1`, `as->as_npages2`, `as->as_stackpbase`, aventi i seguenti valori: 0x200000, 0x300000, 0x3000, 0x7000, 3, 4, 0x400000. Le pagine in OS161 hanno una dimensione di 4 KB. È anche noto che a uno stack user vengono allocate 18 pagine in `dumbvm`.

Dati i seguenti indirizzi logici (potrebbero essere sia user che kernel) convertirli nei relativi indirizzi fisici:

- 0x8110
- 0x6500
- 0x7FFFE010
- 0x805000B0

Domanda 5 (Syscalls e Debugging Lab4)

Q8) Funzione `sys_getpid`:

Considerare la funzione `sys_getpid` (prototipo sotto):

```
pid_t sys_getpid(void);
```

Implementato in Lab4. Per ciascuna delle seguenti frasi, rispondere vero/falso:

- La funzione è necessaria per implementare `sys_waitpid`, perché è necessario il pid del processo in attesa.

- La funzione è necessaria per implementare la chiamata di sistema **getpid**, e sarà chiamata dal kernel, nella funzione **syscall**.
- La funzione è necessaria per implementare la chiamata di sistema **getpid** e verrà chiamata direttamente da un processo utente.
- La funzione restituisce il pid del processo corrente.
- La funzione viene chiamata per ottenere il pid di un altro processo (quello di cui attendere la terminazione).
- La funzione viene chiamata da **sys_exit**, in modo da conoscere il pid del processo a cui inviare una segnalazione.

Q9) Debugging `proc_wait`:

Viene riportata una versione della funzione `proc_wait`, fornita all'interno della soluzione proposta nel laboratorio 4. La funzione contiene 3 errori. Trovarli e, per ognuno di essi, fornire una breve spiegazione (perché sono errori?).

```
int proc_wait(struct proc *proc) {
    int return_status;
    KASSERT (proc != NULL);
    KASSERT (proc != kproc);

    cv_wait(proc->p_cv, proc->p_lock);
    lock_acquire (proc->p_lock);
    return_status = proc->p_status;

    if (return_status < 0)
        return return_status;

    lock_release(proc->p_lock);
    proc_destroy (curproc);
    return return_status;
}
```